AirSunBox
MONITOREO AMBIENTAL

Diseño de arquitecturas iniciales

Sistema de monitoreo de calidad del aire en tiempo real

Dany de León Santizo
Juan Carlos Rodríguez López

Martes, 18 de agosto, 2026
Quetzaltenango

Introducción

Este documento describe el diseño, estructuración y despliegue de la arquitectura tecnológica e infraestructura que sustenta la plataforma de monitoreo de calidad del aire en tiempo real. Cada sección detalla el objetivo, función y configuración de los componentes de hardware y software utilizados, operando bajo un modelo de infraestructura en la nube pública, alojado en el proveedor cloud seleccionado.

Se expone a detalle la interacción entre la capa de presentación (Frontend desacoplado), el núcleo de procesamiento y orquestación mediante contenedores (Backend, Broker MQTT, Base de Datos y Caché), así como los flujos de comunicación IoT, las estrategias de integración y despliegue continuo (CI/CD), y las políticas de respaldo automatizado que garantizan la alta disponibilidad y seguridad del sistema.

Diseño Inicial de la Arquitectura de Software

El sistema ha sido diseñado bajo un enfoque de sistemas distribuidos, separando claramente las responsabilidades entre el cliente (Frontend), el servidor lógico (Backend) y el gestor de telemetría (Broker). A nivel de código, se implementan los siguientes patrones arquitectónicos:

Arquitectura del Backend (Fastify / Node.js)

El backend principal (API REST) utiliza un patrón de Arquitectura en Capas (Layered Architecture), lo que permite desacoplar la lógica de negocio del acceso a la base de datos y la recepción de peticiones HTTP:

- **Capa de Rutas (Routes):** Define los endpoints (ej. /api/devices, /api/organizations) y valida los esquemas de entrada.
- **Capa de Controladores (Controllers):** Orquesta el flujo de la petición, maneja las respuestas HTTP y los códigos de estado.
- **Capa de Servicios (Services):** Contiene la lógica de negocio pura del sistema (ej. cálculos de promedios de calidad del aire, validación de reglas 2FA).
- **Capa de Acceso a Datos (Repositories / ORM):** Gestiona las consultas e interacciones directas con TimescaleDB y Redis.

Arquitectura del Frontend (React + Vite)

La interfaz de usuario se rige por una Arquitectura Basada en Componentes.

- La interfaz se divide en piezas modulares y reutilizables (ej. SensorCard, ChartWidget, Sidebar).
- Se implementa un gestor de estado global (Zustand) para mantener la consistencia de la sesión del usuario y las preferencias de interfaz a través de las distintas vistas de la plataforma, mientras que la actualización de datos ambientales en tiempo real se gestiona mediante conexiones WebSocket.

Patrones de Comunicación y Flujo de Datos

El software utiliza tres paradigmas de comunicación para asegurar eficiencia:

1. **Patrón REST (Cliente-Servidor):** Utilizado para operaciones CRUD (Crear, Leer, Actualizar, Borrar) convencionales desde el panel web hacia la base de datos (ej. registrar un nuevo usuario).
2. **Patrón Publicar/Suscribir (Pub/Sub):** Mediante el protocolo MQTT. Los dispositivos ESP32 *publican* lecturas, y el backend se *suscribe* a estos canales para procesar la telemetría sin saturar la red con peticiones HTTP innecesarias.
3. **Comunicación Bidireccional (WebSockets):** Mantiene una conexión abierta entre el navegador del cliente y el servidor Fastify para "empujar" las alertas críticas de calidad del aire en milisegundos, sin que el usuario deba recargar la página.



Diagrama mas detallado en Diagramas de Arquitectura

Arquitectura Cloud Azure virtual machine

(Propuesta 1)

Capa de Presentación (Frontend Desacoplado)

- **Netlify**: Servicio de *hosting* estático global. Almacena y distribuye la aplicación cliente construida en React + Vite. Al estar desacoplado del servidor principal, reduce la carga computacional en Azure y garantiza tiempos de respuesta óptimos para el usuario final mediante su Red de Distribución de Contenidos (CDN).

Núcleo de Procesamiento (Azure Cloud)

El procesamiento principal reside en una Azure Virtual Machine equipada con el sistema operativo Linux (Ubuntu 22.04 LTS). Para asegurar la modularidad, todos los servicios internos se ejecutan mediante Docker Compose, creando un entorno aislado.

- **Proxy Inverso (Nginx):** Funciona como la puerta de enlace (Gateway) del servidor. Gestiona los certificados de seguridad SSL/TLS (Let's Encrypt) y expone únicamente los puertos necesarios: el puerto web seguro (443 – HTTPS) y el puerto de telemetría IoT (8883 – MQTTS).
- **Broker MQTT (EMQX):** Motor de mensajería encargado de recibir y autenticar las lecturas de calidad del aire transmitidas por los microcontroladores ESP32 en tiempo real.
- **API Backend (Fastify):** Es el servidor lógico de la plataforma. Se suscribe al broker EMQX para procesar la telemetría, expone rutas REST para el panel de administración web, gestiona las conexiones persistentes por *WebSockets* (WSS) y maneja la autenticación segura (JWT + 2FA).

- **Caché y Memoria (Redis):** Almacenamiento rápido en memoria utilizado para la gestión de sesiones de usuarios y el refresco en tiempo real del estado de los sensores.
- **Base de Datos (TimescaleDB):** Motor de base de datos relacional (extensión de PostgreSQL) optimizado específicamente para almacenar y consultar grandes volúmenes de datos de series de tiempo (Time-Series), ideal para el registro histórico de las estaciones IoT.

Automatización, Respaldo y Servicios Externos

- **Gestión de Backups (Azure Blob Storage):** Se implementa un *script* automatizado que genera respaldos de la base de datos (mediante pg_dump) y los exporta a un contenedor de almacenamiento en bloque (*Blob Storage*) en Azure, garantizando la integridad y recuperación de datos ante desastres.

- **Despliegue Continuo (GitHub Actions):** Pipeline de CI/CD que, al detectar una integración exitosa en la rama principal (main), se conecta vía SSH al servidor de Azure, descarga la última versión de la imagen Docker y reinicia los contenedores de forma automatizada.

- **Servicio SMTP:** Integración con un proveedor de correos de terceros (ej. SendGrid o Resend) para el envío de códigos de autenticación de dos factores (2FA) y recuperación de credenciales.

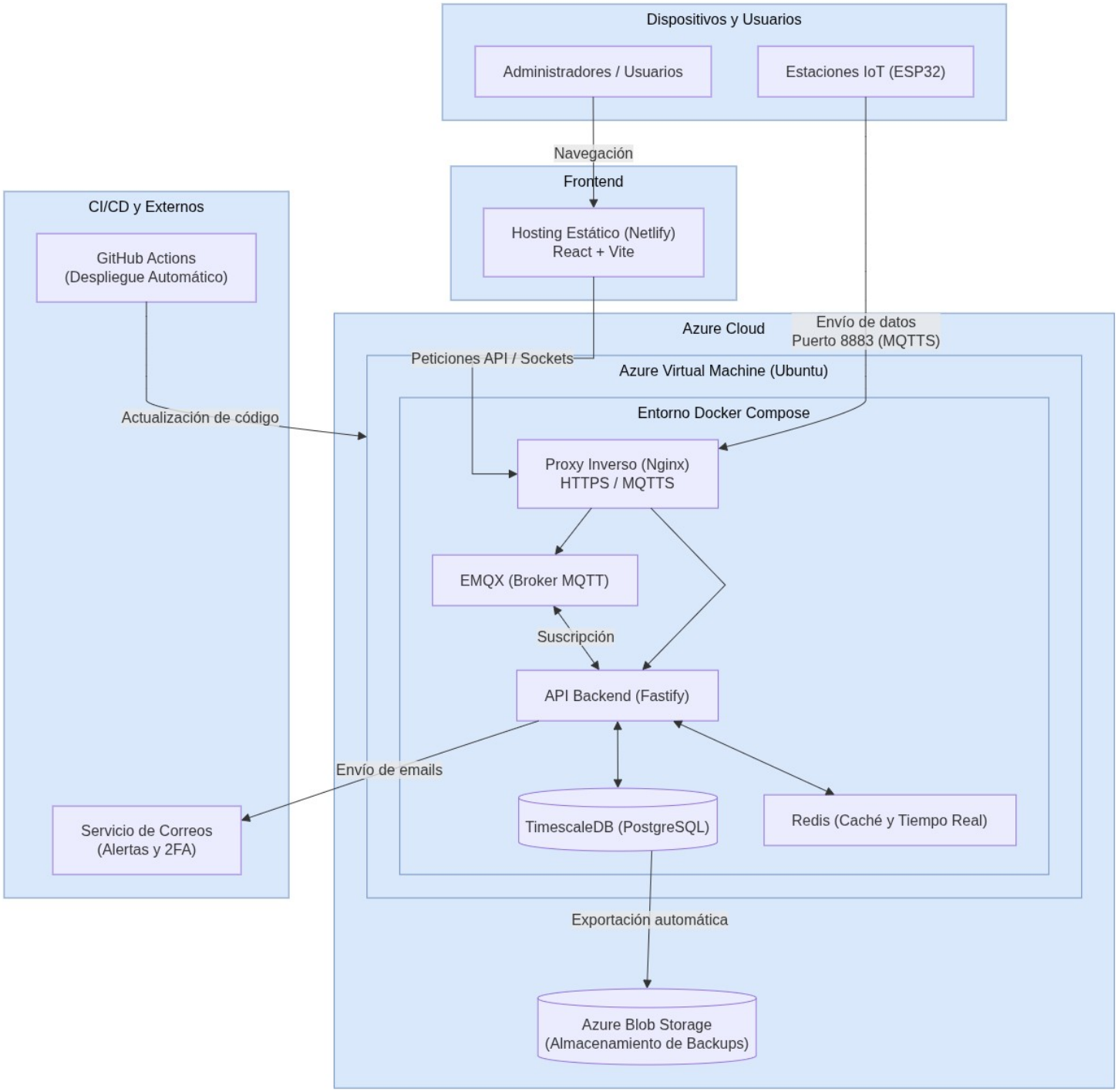


Diagrama mas detallado en Diagramas de Arquitectura

Arquitectura Cloud AWS

(Propuesta 2)

Capa de Presentación (Frontend Desacoplado)

- **Netlify**: Servicio de *hosting* estático global. Almacena y distribuye la aplicación cliente construida en React + Vite. Al estar desacoplado del servidor principal, reduce la carga computacional en el servidor principal y garantiza tiempos de respuesta óptimos para el usuario final mediante su Red de Distribución de Contenidos (CDN).

Núcleo de Procesamiento (AWS Cloud EC2)

El procesamiento principal reside en una Instancia Amazon EC2 (Elastic Compute Cloud) equipada con el sistema operativo Linux (Ubuntu 22.04 LTS). Para asegurar la modularidad, todos los servicios internos se ejecutan mediante Docker Compose, creando un entorno aislado.

- **Proxy Inverso (Nginx):** Funciona como la puerta de enlace (Gateway) del servidor. Gestiona los certificados de seguridad SSL/TLS (Let's Encrypt) y expone únicamente los puertos necesarios: el puerto web seguro (443 – HTTPS) y el puerto de telemetría IoT (8883 – MQTTS).
- **Broker MQTT (EMQX):** Motor de mensajería encargado de recibir y autenticar las lecturas de calidad del aire transmitidas por los microcontroladores ESP32 en tiempo real.
- **API Backend (Fastify):** Es el servidor lógico de la plataforma. Se suscribe al broker EMQX para procesar la telemetría, expone rutas REST para el panel de administración web, gestiona las conexiones persistentes por *WebSockets* (WSS) y maneja la autenticación segura (JWT + 2FA).

- **Caché y Memoria (Redis):** Almacenamiento rápido en memoria utilizado para la gestión de sesiones de usuarios y el refresco en tiempo real del estado de los sensores.

- **Base de Datos (TimescaleDB):** Motor de base de datos relacional (extensión de PostgreSQL) optimizado específicamente para almacenar y consultar grandes volúmenes de datos de series de tiempo (Time-Series), ideal para el registro histórico de las estaciones IoT.

Automatización, Respaldo y Servicios Externos

- **Gestión de Backups (Amazon S3):** Se implementa un script automatizado que genera respaldos de la base de datos (mediante pg_dump) y los exporta a un bucket de almacenamiento en **Amazon S3**, garantizando la integridad y recuperación de datos ante desastres.

- **Despliegue Continuo (GitHub Actions):** Pipeline de CI/CD que, al detectar una integración exitosa en la rama principal (main), se conecta vía SSH al servidor de AWS, descarga la última versión de la imagen Docker y reinicia los contenedores de forma automatizada.

- ***Servicio SMTP:*** *Integración con un proveedor de correos de terceros (ej. SendGrid o Resend) para el envío de códigos de autenticación de dos factores (2FA) y recuperación de credenciales.*



Diagrama mas detallado en Diagramas de Arquitectura